Document number	P2472R2
Date	2022-03-08
Reply-to	Jarrad J. Waterloo <> Zhihao Yuan <>
Audience	Library Evolution Working Group (LEWG)

make `function_ref` more functional

Table of contents

Changelog

R1

- Moved from my make_function_ref implementation to Zhihao Yuan's nontype implementation
  - Constructors were always my ideal over factory function but I was unable to make it work.
  - nontype has better type deduction

- Most of the changes were syntactical and reasoning based on feedback as well as making examples even more concise

R2

- Added this changelog
- Transformed solution into the wording
- Added a third constructor which takes a pointer instead of a reference for convenience
- Revised example from cat to rule/test
- Added deduction guide

Abstract

This document proposes adding additional constructors to `function_ref` [1] in order to make it easier to use, more efficient, and safer to use with common use cases.

Currently, a `function_ref`, can be constructed from a lambda/functor, a free function pointer, and a member function pointer. While the lambda/functor use case does support type erasing `this` pointer, its free/member function pointer constructors do **not** allow type erasing any arguments, even though these two use cases are common.

`function_ref`

	Stateless	Stateful
Lambda/Functor	✓	✓
Free Function	✓	✗
Member Function	✓	✗

Motivating Examples

Given

```
struct rule {
    void when() {
    }
};
```

```
// rule could just as easily be test which also have when and then
        functions

void then(rule& r) {
}

void rule_when(rule& r) {
    r.when();
}

struct callback {
    rule* r;
    void (*f)(rule&);
};

rule localTaxRule;
```

member/free function with type erasure

Since typed erased free and member functions are not currently supported, the current `function_ref` proposal forces users to create an unnecessary temporary functor, likely with `std::bind_front` or a stateful/capturing lambda that the developer hopes the optimizer to elide.

member function with type erasure

C/C++ core language	`callback cb = {&localTaxRule, [](rule& r) {r.when();}};`
	`// or`
	`callback cb = {&localTaxRule, rule_when};`

function_ref	`// separate temp to prevent dangling`
	`// when temp is passed to multiple arguments`
	`auto temp = [&localTaxRule](){localTaxRule.when();};`
	`function_ref<void()> fr = temp;`
	`// or when given directly as a function argument`
	`some_function([&localTaxRule]() {localTaxRule.when();});`

proposed	`function_ref<void()> fr = {nontype<&rule::when>, localTaxRule};`

free function with type erasure

C/C++ core language	`callback cb = {&localTaxRule, [](rule& r) {then(r);}};`
	`// or`
	`callback cb = {&localTaxRule, then};`

function_ref	`// separate temp to prevent dangling`
	`// when temp is passed to multiple arguments`
	`auto temp = [&localTaxRule]() {then(localTaxRule);};`
	`function_ref<void()> fr = temp;`
	`// or when given directly as a function argument`
	`some_function([&localTaxRule]() {then(localTaxRule);});`

proposed	`function_ref<void()> fr = {nontype<then>, localTaxRule}`

This has numerous disadvantages compared to what the C++ core language can do. It is inefficient, hard to use, and unsafe to use.

Not easy to use

- Unlike the direct initialization of the C/C++ core language example, the initialization of `function_ref` is bifurcated by passing it directly as a function argument or using 2-step initialization by first creating a named temporary. Direct initializing a variable of `function_ref` is needed if the same object serves as multiple arguments on either the same but more likely different functions. However, this leads to immediate dangling, which must introduce an extra line of code. Furthermore, getting in the habit of only passing `function_ref` as an argument to a function results in users duplicating lambdas when needed to be used more than once, again needless increasing the code volume. Initialization is consistently safe in both the C/C++ core language and the proposed revision, regardless of whether `function_ref` appears as a variable.

Why should a stateful lambda even be needed when the signature is compatible?

- The C/C++ core language example works with a “stateless” lambda. A new anonymous type is created with states when requiring a “stateful” lambda. That

state's lifetime also needs to be managed by the user.
- When using a lambda, the user must manually forward the function arguments. However, in the C/C++ core language example, the syntax is straightforward when the function exists. We simply pass a function pointer to the callback struct.

Inefficient

- By requiring a "stateful" lambda, a new type is created with a lifetime that must be managed. `function_ref` is a reference to a "stateful" lambda which also is a reference. Even if the optimizer can remove the cost, the user is still left with the burden in the code.
- In the proposed, `nontype` does not take a pointer but rather a member function pointer initialization statement thus giving the compiler more information to resolve the function selection at compile time rather than runtime, granted it is more likely with free functions instead of member functions.

Unsafe

- Direct initialization of `function_ref` outside of a function argument immediately dangles. Some would say that this is no different than other standardized types such as `string_view`. However, this is not true.

C/C++ core language	`std::string_view sv = "hello world"s;// immediately dangling`
	`auto c = "hello world"s;`
	`std::string_view sv = c;// DOES NOT DANGLE`

function_ref	`function_ref<void()> fr = [&localTaxRule]() {localTaxRule.when();};// immediately dangling`
	`// or`
	`function_ref<void()> fr = [&localTaxRule]() {then(localTaxRule);};// immediately dangling`

proposed	`function_ref<void()> fr = {nontype<&rule::when>, localTaxRule};// DOES NOT DANGLE`
	`// or`
	`function_ref<void()> fr = {nontype<then>, localTaxRule}// DOES NOT DANGLE`

While both the original `function_ref` proposal and the proposed addendum perform the same desired task, the former dangles and the later doesn't. It is clear from the immediately dangling `string_view` example that it dangles because `sv` is a reference and `""s` is a temporary. However, it is less clear in the original `function_ref` example. While it is true and clear that `fr` is a reference and the stateful lambda is a temporary. It is not what the user of `function_ref` is intending or wanting to express. `localTaxRule` is the state that the user wants to type erase and `function_ref` would not dangle if `localTaxRule` was the state since it is not a temporary and has already been constructed higher up in the call stack. Further, the member function `when` or the free function `then` should not dangle since functions are stateless and also global. So member/free function with type erasure use cases are more like `string_view` when the referenced object is safely constructed.

	easier to use	more efficient	safer to use
C/C++ core language	✓	✓	✓
function_ref	✗	✗	✗
proposed	✓	✓	✓

What is more, member/free functions with type erasure are common use cases! "Member functions with type erasure" is used by delegates/events in object oriented programming and "free functions with type erasure" are common with callbacks in procedural/functional programming.

member function without type erasure

The fact that users do not expect "stateless" things to dangle becomes even more apparent with the member function without type erasure use case.

C/C++ core language

```
void (rule::*mf)() = &rule::when;
```

function_ref

```
// separate temp needed to prevent dangling
// when temp is passed to multiple arguments
auto temp = &rule::when;
function_ref<void(rule&)> fr = temp;
// or when given directly as a function argument
some_function(&rule::when);
```

proposed	`function_ref<void(rule&)> fr = {nontype<&rule::when>};`

Current `function_ref` implementations store a reference to the member function pointer as the state inside `function_ref`. A trampoline function is required regardless. However, the user **expected** behavior is for `function_ref` referenced state to be unused/`nullptr`, as **all** of the arguments must be forwarded since **none** are being type erased. Such dangling is **never** expected, yet the current `function_ref` proposal/implementation does. Similarly, this use case suffers, just as the previous two did, with respect to ease of use, efficiency, and safety due to the superfluous lambda/functor and two-step initialization.

	easier to use	**more efficient**	**safer to use**
C/C++ core language	✓	✓	✓
function_ref	✗	✗	✗
proposed	✓	✓	✓

free function without type erasure

The C/C++ core language, `function_ref`, and the proposed examples are approximately equal concerning ease of use, efficiency, and safety for the free function without type erasure use case. While the proposed `nontype` example is slightly wordier because of the template `nontype`, it is more consistent with the other three use cases, making it more teachable and usable since the user does not need to choose between bifurcation practices. Also, the expectation of unused state and the function being selected at compile time still applies here, as it does for member function without type erasure use case.

C/C++ core language	`void (*f)(rule&) = then;`
function_ref	`function_ref<void(rule&)> fr = then;`
proposed	`function_ref<void(rule&)> fr = {nontype<then>};`

	easier to use	**more efficient**	**safer to use**
C/C++ core language	✓	✓	✓
function_ref	✓	✓	✓

	easier to use	more efficient	safer to use
proposed	✓	✓	✓

Remove existing free function constructor?

Should the existing free function constructor be removed with the overlap in functionality with the free function without type erasure use case? NO. Initializing a `function_ref` from a function pointer instead of a non-type function pointer template argument is still usable in the most runtime of libraries, such as dynamically loaded libraries where function pointers are looked up. However, it is not necessarily the general case where users work with declarations found in header files and modules.

Wording

The wording is relative to P0792R8.

Add new templates to 20.2.1 [utility.syn], header `<utility>` synopsis after `in_place_index_t` and `in_place_index`:

```
namespace std {
  [...]

  // nontype argument tag
  template<auto V>
    struct nontype_t {
      explicit nontype_t() = default;
    };

  template<auto V> inline constexpr nontype_t<V> nontype{};
}
```

Add a definition to 20.14.3 [func.def]:

```
template<auto f> constexpr function_ref(nontype_t<f>) noexcept;
```

> *Constraints:* `is-invocable-using<decltype(f)>` is `true`.
>
> *Effects:* Initializes `bound-entity` with `nullptr`, and `thunk-ptr` to address of a function such that `thunk-ptr(bound-entity, call-args...)` is expression equivalent to `invoke_r<R>(f, nullptr, call-args...)`.

```
template<auto f, class T> function_ref(nontype_t<f>, T& state)
        noexcept;
```

> *Constraints:* `is-invocable-using<decltype(f), cv T&>` is true.
>
> *Effects:* Initializes `bound-entity` with `addressof(state)`, and `thunk-ptr` to address of a function such that `thunk-ptr(bound-entity, call-args...)` is expression equivalent to `invoke_r<R>(f, static_cast<T cv&>(bound-entity), call-args...)`.

```
template<auto f, class T> function_ref(nontype_t<f>, cv T* state)
        noexcept;
```

> *Constraints:* `is-invocable-using<decltype(f), cv T*>` is `true`.
>
> *Effects:* Initializes `bound-entity` with `state`, and `thunk-ptr` to address of a function such that `thunk-ptr(bound-entity, call-args...)` is expression equivalent to `invoke_r<R>(f, static_cast<cv T*>(bound-entity), call-args...)`.

```
  template<class R, class... ArgTypes>
  class function_ref<R(ArgTypes...) cv ref noexcept(noex)> {
  public:
    // [func.wrap.ref.ctor], constructors ...
    ...
    template<auto f> constexpr function_ref(nontype_t<f>) noexcept;
    template<auto f, class T> function_ref(nontype_t<f>, T&) noexcept;
    template<auto f, class T> function_ref(nontype_t<f>, cv T*)
        noexcept;

    ...
  };

  // [func.wrap.ref.deduct], deduction guides
  template<auto f>
    function_ref(nontype_t<f>) ->
        function_ref<std::remove_pointer_t<decltype(f)>;
  template<auto f>
    function_ref(nontype_t<f>, auto) -> function_ref<see below>;
}
```

> **Deduction guides**
>
> [func.wrap.ref.deduct]

```
template<auto f>
  function_ref(nontype_t<f>) ->
        function_ref<std::remove_pointer_t<decltype(f)>;
```

> *Constraints:* `is_function_v<f>` is `true`.

```
template<auto f>
  function_ref(nontype_t<f>, auto) -> function_ref<see below>;
```

> *Constraints:* - `decltype(f)` is of the form `R(G::*)(A...) cv &opt noexcept(E)` *for a class type* `G`, *or* - `decltype(f)` *is of the form* `R G::*` for a class type `G`, in which case let `A...` be an empty pack, or - `decltype(f)` is of the form `R(U, A...) noexcept(E)`.
>
> *Remarks:* The deduced type is `function_ref<R(A...) noexcept(E)>`.

Feature test macro

We do not need a feature macro, because we intend for this paper to modify `std::function_ref` before it ships.

Other languages

C# and the .NET family of languages provide this via `delegates` [2].

```
// C#
delegate void some_name();
some_name fr = then;// the stateless free function use case
some_name fr = localTaxRule.when;// the stateful member function use
        case
```

Borland C++ now embarcadero provides this via `__closure` [3].

```
// Borland C++, embarcadero __closure
void(__closure * fr)();
fr = localTaxRule.when;// the stateful member function use case
```

`function_ref` with `nontype` constructors handles all four stateless/stateful free/member use cases. It is more feature-rich than those prior arts.

Example implementation

The most up-to-date implementation, created by Zhihao Yuan, is available on GitHub.[4]

Acknowledgments

Thanks to Arthur O’Dwyer, Tomasz Kamiński, Corentin Jabot and Zhihao Yuan for providing very valuable feedback on this proposal.

References

---

1. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0792r6.html↩
2. https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/using-delegates↩
3. http://docwiki.embarcadero.com/RADStudio/Sydney/en/Closure↩
4. https://github.com/zhihaoy/nontype_functional/blob/main/include/std23/function_ref.h↩